

DSA STUDY BLUEPRINT SERIES

106. DFS Traversal in Graphs

Depth-First Graph Traversal (DFS)

Section 10 • C++ Core Data Structures & Algorithms

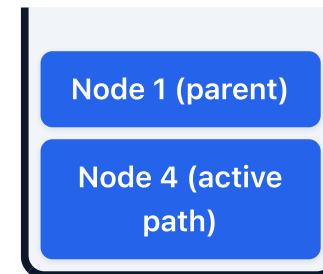
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **DFS:** Explores paths recursively before backtracking.
- Uses a recursion stack or an iterative stack buffer.

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
void dfs(int u, vector<int>& adj, vector<bool>& visited) {  
    visited[u] = true;  
    for (int v : adj[u]) {  
        if (!visited[v]) dfs(v, adj, visited);  
    }  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(V + E)$
Space Complexity	$O(V)$ (Stack depth)

Key Practices & Gotchas

- **Important:** Ideal for connectivity searches and cycle checks.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace Dijkstra distance relaxation updates on active vertices:

Vertex popped	Adjacent edge checked	Current distance value	New path calculation	Relaxation action
U (Dist = 2)	U → V (Weight = 3)	V (Dist = 7)	New Dist = 2 + 3 = 5	Relax: Update V (Dist = 5)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Shortest Path choices:** BFS finds unweighted short paths; Dijkstra handles positive weights; Bellman-Ford resolves negative edges.
- **Spanning Trees:** Prim's builds MST from a start node; Kruskal's sorts all edges using disjoint sets.
- **Related LeetCode Problems:** #200 Number of Islands, #743 Network Delay Time, #787 Cheapest Flights Within K Stops.